

Autonomous Systems Lab - Documentation

Moez Mufti, Muhammad Ali, Muhammad Hamas¹

Abstract: This report documents Team 1's autonomous systems lab project: an SVM-based line-following pipeline and a truck-platooning communication protocol. Annotated CARLA camera frames were used to train a Support Vector Regressor for continuous steering. Platooning is specified across three scenarios, coupling, decoupling, and communication failure, each modelled with SysML requirements and UML state machines, then formalised and verified as timed automata in UPPAAL. Verification establishes deadlock freedom, bounded timing, and safe handling of concurrent join, leave, and failure events.

1 Motivation

Autonomous trucks operating in coordinated platoons promise reduced fuel consumption, lower driver workload, and better road throughput, but only if the underlying control and communication logic can be trusted under both normal and faulty conditions. This project addresses two building blocks of such a system: a vision-based line-following controller that keeps a single vehicle on its lane, and a set of platoon coordination protocols that let trucks join, leave, and recover from communication loss without violating safety constraints. Rather than relying on informal reasoning, the platoon protocols are specified with SysML and UML models and then checked as timed automata in UPPAAL, so that timing bounds and safety properties are verified against every reachable execution rather than a handful of manually constructed test cases [Syshandb2000].

2 Data Collection, Annotation, and SVM Training

2.1 Data Collection

Training data was recorded as ROS bag files while driving through the CARLA simulator. Camera frames were extracted from these recordings at regular intervals, giving a mixed set of straight sections, gradual bends, and sharp turns. Backgrounds were deliberately left unfiltered: buildings, foliage, and road markings all remain visible in the frame, so the model is trained against realistic clutter rather than a clean synthetic scene.

¹ moez.mufti@hshl.de, muhammad.ali@stud.hshl.de, muhammad.hamas@stud.hshl.de

2.2 Manual Annotation

Each selected frame was manually annotated by drawing a green path over the visible lane, from the vehicle position toward the expected lane centre further ahead. Straight roads produced an almost vertical path, gradual bends produced a moderate lateral curve, and sharp bends produced a strong deflection. Annotation was kept consistent by using approximately the same starting position and look-ahead distance for every frame, and by following the intended lane centre rather than nearby vehicles or road edges. Ambiguous or obstructed frames were reviewed and corrected by hand before being accepted into the dataset.

2.3 Feature Extraction

Each annotated frame is reduced to a fixed-size feature vector before training. The HSV saturation channel is boosted to 150 percent so that the green line remains separable under varying simulated lighting. A colour threshold then produces a binary mask of the line, which is constrained to the bottom 50 percent and centre 70 percent of the frame to remove far-field pixels that are not relevant to the immediate steering decision. Where the threshold produces more than one disconnected segment, for example at a fork, the blob closest to the vehicle is kept and the rest are discarded. The resulting mask is resized to 64 by 64 pixels and flattened into a 4096 dimensional binary feature vector.

2.4 Label Generation

The manually drawn path is converted into a single signed steering value per frame. Two quantities are computed from the annotated line: a heading term, taken from the horizontal displacement between the top and bottom of the path, and an offset term, taken from the horizontal displacement of the path's base from the frame centre. The two terms are combined into one label, scaled and clipped to the range $[-1.0, 1.0]$. Values close to zero are read as *Straight*, a moderate magnitude as a slight left or right correction, and a larger magnitude as a hard turn. Figure 7 shows one annotated frame next to the resulting steering readout: a value of +0.794 corresponds to a hard right correction, matching the strong curvature of the drawn path.

2.5 Model Training

The labelled feature vectors are used to train a Support Vector Regressor inside a scikit-learn pipeline: a StandardScaler followed by an SVR with an RBF kernel, $C = 1.0$ and $\epsilon = 0.1$. The epsilon-insensitive loss means that errors smaller than ϵ are not

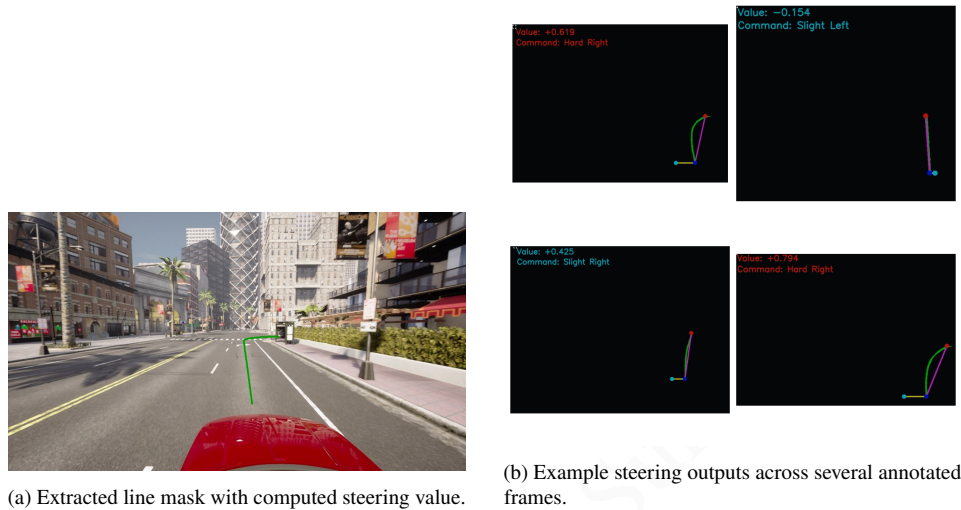


Fig. 1: CARLA raw image and annotated steering result.

penalised, so the model does not fit to pixel-level noise in the mask. Regression was chosen over classification because steering is a continuous physical quantity: a classifier would force the output into fixed buckets and produce abrupt corrections at the class boundaries, while the SVR produces a smooth value across the whole range.

At inference time the raw per-frame prediction is still noisy from one frame to the next, so a five-frame moving average is applied before the value is used for steering. The discrete labels shown in Figure 7 (Straight, Slight Right, Hard Right, and so on) are derived from this smoothed value only for logging purposes and play no part in the control loop itself.

3 Scenario 1: Coupling

3.1 Scenario Overview

Truck platooning allows independent trucks to form a synchronized convoy via V2V communication, reducing drag, fuel use, and driver workload. This scenario covers only the join manoeuvre itself: a lone truck requesting entry into an already-moving convoy, not steady-state coordinated driving.

The joining truck, while driving independently, detects a convoy and sends a join request to the lead truck, which is authorised via a cloud-based network operations centre (NOC) check. Once authorised, a bounded-latency V2V link is established and follower trucks are notified of the incoming join. A safety check then verifies the inter-vehicle gap, lane

conditions, and blind spots; failing this check, or a timeout/link loss at any earlier step, aborts the manoeuvre back to independent driving. If the check passes, the truck synchronises speed with the lead truck via ACC at 50 Hz, performs a final lane/spacing correction, and merges into its assigned slot, becoming an active platoon member.

3.2 Requirements Model

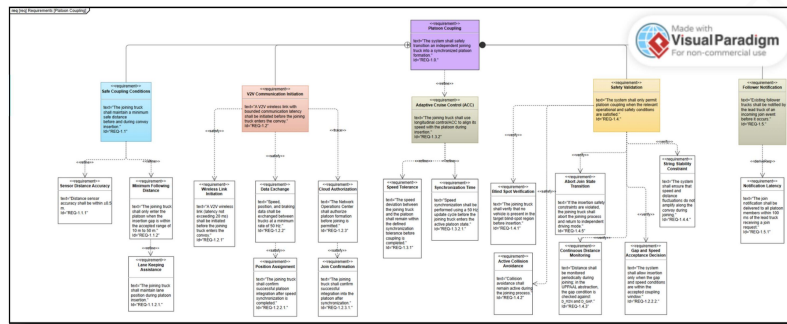


Fig. 2: Requirements diagram for the Coupling scenario.

Figure 2 decomposes the top-level requirement; safe transition of an independent truck into a synchronized platoon into five clusters: **Safe Coupling Conditions** (minimum safe distance before/during insertion, refined by sensor accuracy and lane-keeping sub-requirements); **V2V Communication Initiation** (bounded-latency link established before entry, with data exchanged at ≥ 50 Hz and cloud authorization confirmed after synchronization); **Adaptive Cruise Control** (speed aligned with the platoon within tolerance, synchronized within a bounded number of cycles); **Safety Validation** (blind-spot verification, an abort transition on constraint violation, collision avoidance, and a string-stability constraint preventing amplified fluctuations); and **Follower Notification** (existing members notified within 100 ms of the join request).

3.3 State-Based Behaviour

Figure 3 shows the controller's behavioural states: **Independent Driving** (initial; sends the join request on convoy detection), **Cloud Check (NOC)** (authorisation accept/deny), **Insertion Readiness Check** (V2V link setup and follower notification, bounded by a timeout), **Safety Check** (lane, distance, speed, and blind-spot feasibility tests), **Speed Synchronization** (50 Hz data exchange, ACC speed adjustment), **Platoon Alignment** (final lane/spacing correction), **Joining Truck Insertion** (physical merge into the convoy slot), and **Platoon Active** (terminal state; coordinated operation with the lead truck as reference). Any failed check, timeout, or emergency abort returns the truck to Independent Driving.

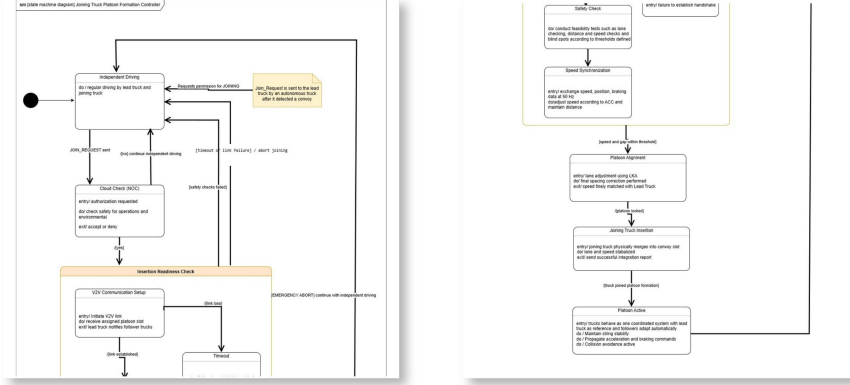


Fig. 3: State machine of the joining-truck platoon-formation controller.

3.4 Message-Sequence Behaviour

The joining truck sends `join_req!` from Independent; the lead truck answers `join_appr!` and begins its `HeartBeat_Emit` loop, at which point its role in the manoeuvre is complete. The joining truck alone determines the outcome from here: `SafetyCheck` evaluates the gap against $[D_{\min}, D_{\text{gap}}]$, aborting to Independent if it falls outside that window or proceeding to `SpeedSync` otherwise. `SpeedSync` repeats for three rounds, narrowing the gap each pass, before transitioning to `PlatoonActive` with `jt_coupled` set to true.

3.5 Joining-Truck Timed Automaton

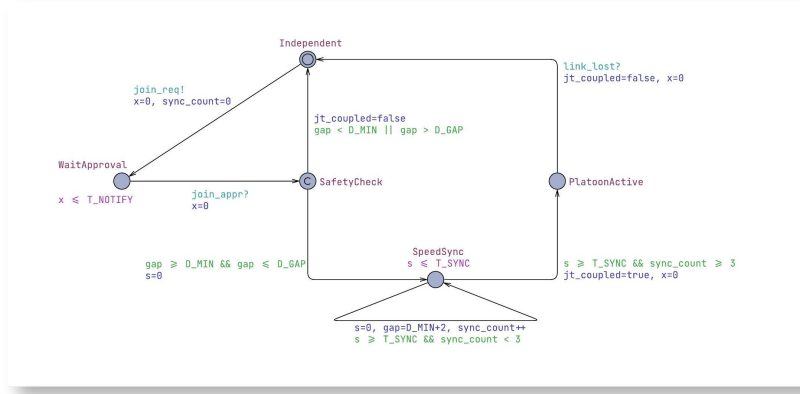
The joining truck drives the coupling manoeuvre of Scenario 1 (Section 3) through five locations: Independent (initial), `WaitApproval`, `SafetyCheck` (committed), `SpeedSync`, and `PlatoonActive` using clocks `x`, `s` and counter `sync_count`.

From Independent, `join_req!` resets `x` and `sync_count` and moves to `WaitApproval`, whose invariant `x ≤ T_NOTIFY` bounds the wait for `join_appr?`. The committed `SafetyCheck` state then fires immediately on one of two mutually exclusive edges:

```

gap < D_MIN || gap > D_GAP  -> jt_coupled = false  (abort to Independent)
gap >= D_MIN && gap <= D_GAP -> s = 0             (proceed to SpeedSync)
  
```

directly encoding the Safety Validation requirement cluster of Figure 2. `SpeedSync` (invariant `s ≤ T_SYNC`) loops while `sync_count < 3`, each round nudging the gap toward its target; the bound guarantees termination. On the third round it transitions to `PlatoonActive` and sets `jt_coupled = true`. The remaining edge, `link_lost?` back to Independent, belongs



JOINING TRUCK

Fig. 4: Timed automaton of the joining truck.

to the communication-failure scenario rather than Coupling, it appears here only because the template is reused unmodified across scenarios.

4 Scenario 2: Decoupling

A truck platoon must support the safe departure of individual members without endangering the leaving truck, the remaining platoon, or surrounding traffic. A truck may leave because of a route change, delivery stop, refuelling stop, or another operational requirement. During this manoeuvre, coordinated platoon control is gradually replaced by independent vehicle control.

The decoupling process is safety-critical because a departure gap must first be created, the adjacent lane must be verified as clear, and collision avoidance and lane-keeping functions must remain active throughout the lane change. At the same time, the remaining trucks must reorganise their positions, spacing, and speed after the departing truck has left.

For this reason, the scenario contains explicit requirements for request authorization, communication, gap creation, safe departure validation, and platoon reconfiguration.

4.1 Overview of the Decoupling Scenario

The decoupling scenario begins while the leaving truck is in the *Platoon Active* state. When a departure reason is detected, the truck sends a `LEAVE_REQUEST` to the lead truck and waits

for LEAVE_APPROVAL. If required by the fleet policy, the request may also be checked by the Network Operations Center.

After approval, the truck remains in its current platoon position while the following vehicles create a departure gap. The following trucks reduce their speed until the separation reaches at least 50 m. The gap must be established within five seconds. If the gap cannot be created before the timeout, the departure is cancelled and the truck remains in the platoon.

Once the required gap is available, the system performs a safety verification. Blind-spot monitoring, environmental sensing, lane-keeping assistance, forward-collision warning, and collision avoidance are checked before the departure manoeuvre is started.

The leaving truck then exits its platoon position and changes lane. During this manoeuvre, lane-keeping assistance maintains the departure trajectory, adaptive cruise control transitions to independent speed control, and automatic emergency braking and collision warning remain active.

After the truck has separated, two activities take place:

- the leaving truck enters independent driving and sends DEPARTURE_COMPLETE; and
- the remaining trucks reorganise the platoon by updating position identifiers, restoring spacing, and re-synchronising speed.

The scenario ends when the leaving truck is operating independently and the remaining platoon has been locked into its new formation.

4.2 Requirements View

The requirement diagram divides platoon decoupling into five main branches.

Leave Request and Authorization. The leaving truck shall send a LEAVE_REQUEST to the lead truck before departure. The lead truck shall acknowledge the request, while fleet or NOC authorization may additionally be required depending on the operating policy.

Communication. All platoon members shall be informed of the departure. The lead truck broadcasts the departure notification, and the leaving truck later sends a departure confirmation. Departure-related notifications shall be delivered within 100 ms.

Gap Creation. Following vehicles shall adjust their speed to create the departure gap. The required separation shall be at least 50 m, and the gap shall be established within five seconds.

Safe Departure. The leaving truck shall only begin the lane-change manoeuvre when the adjacent lane, blind-spot region, speed conditions, and available gap have been verified as safe. Lane keeping, environmental monitoring, active collision avoidance, and string stability shall remain active during departure.

Platoon Reconfiguration. After the truck leaves, the remaining vehicles shall update their platoon positions, restore inter-vehicle spacing, and re-synchronise their speed.

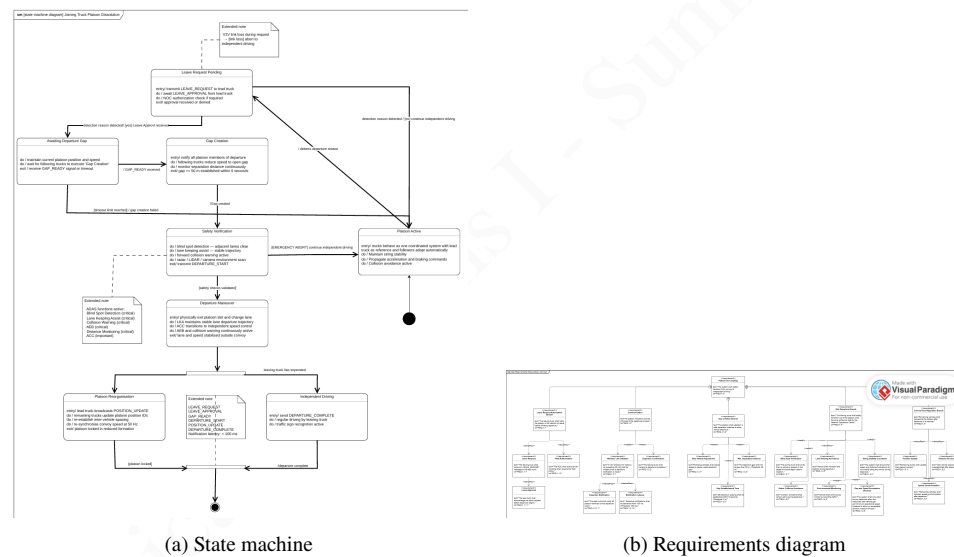


Fig. 5: SysML models for Scenario 2: truck decoupling.

4.3 Behavioural View

Figure 6a shows the behaviour of the leaving truck and the remaining platoon.

Platoon Active. The truck initially operates as a coordinated platoon member. It follows the lead truck and participates in acceleration, braking, string-stability, and collision-avoidance functions.

Leave Request Pending. When a departure reason is detected, the truck sends `LEAVE_REQUEST` and waits for approval from the lead truck. If authorization is denied, the truck remains in the platoon. A V2V link loss during the request phase aborts the procedure and transfers the vehicle to independent operation.

Awaiting Departure Gap and Gap Creation. After approval, the truck maintains its platoon position and speed while the following vehicles reduce speed and create the required gap. A `GAP_READY` event allows the process to continue. If the timeout is reached before the gap is available, the departure is cancelled and the vehicle returns to normal platoon operation.

Safety Verification. Before departure, the controller verifies that the adjacent lane is clear, the blind spot is free, the lane-keeping trajectory is stable, and no collision risk is present. Radar, LiDAR, and camera-based environmental monitoring support this decision. If the safety checks succeed, `DEPARTURE_START` is issued. If they fail, the manoeuvre is aborted and the truck remains in the platoon.

Departure Manoeuvre. The truck physically exits its platoon slot and changes lane. Lane-keeping assistance controls the trajectory, adaptive cruise control changes to independent speed control, and collision warning and automatic emergency braking remain available until the truck is stable outside the convoy.

Independent Driving and Platoon Reorganisation. After separation, the leaving truck enters independent driving and sends `DEPARTURE_COMPLETE`. In parallel, the lead truck broadcasts a `POSITION_UPDATE`. The remaining trucks update their position identifiers, restore spacing, and re-synchronise their speed at 50 Hz. The scenario is complete when both the leaving truck and the reduced platoon have reached stable operating states.

4.4 Formal Model: UPPAAL Mapping and Verified Properties

The UPPAAL model (`clean_UPPAAL_v7_N2.xml`), built with the UPPAAL model checker [`uppaal_tool`, [behrmann2004uppaal](#)], formalises the decoupling behaviour described above so that the timing bounds and safety guards are machine-checked rather than only documented. The relevant templates are **LeadTruck** and **LeavingTruck(id)**, instantiated twice (`lv0`, `lv1`) to model two trucks that may request departure concurrently and contend for the same lead truck via per-truck channels `leave_req[N]`/`leave_appr[N]` – this directly represents the “Following/Leaving Truck vs. Lead Truck” actor split from the use case and sequence diagrams.

SysML state / phase	UPPAAL location	Key guard / invariant
Platoon Active	LeavingTruck.InPlatoon	initial location per truck
Leave Request Pending	LeavingTruck.WaitLeaveAppr	invariant $x \leq T_NOTIFY$ ($10 \triangleq 100$ ms)
Awaiting Departure Gap / Gap Creation	LeavingTruck.GapCreation	invariant $g \leq T_GAP_MAX$; loops while $lgap[id] < D_GAP$
Safety Verification	LeavingTruck.SafetyVerify (committed)	$lgap[id] \geq D_GAP \rightarrow Departed$; else $\rightarrow InPlatoon$ (abort)
Departure Maneuver / Independent Driving	LeavingTruck.Departed	$lv_coupled[id]=false$, $saw_depart=true$
Lead truck authorization	LeadTruck.HandleLeave (committed)	$leave_req[i]? \rightarrow leave_appr[who]!$ (serialised across trucks)

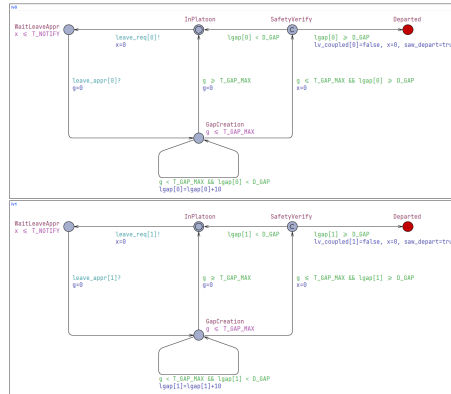
Tab. 1: Correspondence between SysML behavioural states and UPPAAL model elements.

State-to-model correspondence. The gap-creation timeout ($g \geq T_GAP_MAX$) returns the truck to InPlatoon rather than stranding it – the formal counterpart of the state machine’s **[timeout limit reached] / gap creation failed** transition. Similarly, the notification-latency invariant on WaitLeaveAppr formalises the diagrams’ < 100 ms annotation on every leave-related message.

Verified properties (model-checking results). The following queries were checked against the model using UPPAAL’s symbolic model checker [behrmann2004uppaal] (--learning-method 2); all listed properties returned **satisfied**:

- $A[]$ not deadlock – the composed system (JoiningTruck, LeadTruck, two LeavingTrucks, Watchdog) never deadlocks, including under contention between $lv0$ and $lv1$ for the shared lead truck.
- $E\langle \rangle lv0.Departed$ and $E\langle \rangle (lv0.Departed \ \&\& \ lv1.Departed)$ – a single departure, and both concurrent departures, are reachable.
- $A[] (lv0.WaitLeaveAppr \implies lv0.x \leq T_NOTIFY)$ (and the equivalent for $lv1$) – the leave-notification latency bound holds for *both* trucks, i.e. the second truck is not starved while the lead truck serialises requests.
- $lv0.SafetyVerify \dashrightarrow (lv0.Departed \ || \ lv0.InPlatoon)$ – the safety-verification step always resolves to either a completed departure or a safe return to the platoon, never a stuck state.

These results give quantitative backing to the qualitative safety claims in the diagrams: the platoon-wide safety net (fail back to InPlatoon) is not just a documented design intent but a property that holds for every reachable state of the model.



(a) State machine – leaving-truck lifecycle, gap-creation / reorganisation behaviour, and abort conditions.

5 Scenario 3: Communication Failure and Safe Fallback

Truck platooning depends on periodic V2V communication. In normal operation, the lead truck transmits a heartbeat every 100 ms. A watchdog resets its timeout whenever this heartbeat is received. If no heartbeat arrives for 300 ms, the watchdog declares the link lost, broadcasts a failure event, and the affected coupled truck returns to independent driving. Safe fallback must then be reached within 200 ms. During fallback, the onboard controller maintains lane position, monitors distance, enlarges the gap, and activates emergency braking when the minimum safe distance is violated.

5.1 SysML/UML Models

The requirement diagram in Figure 7 groups the scenario into communication monitoring, failure detection, fallback operation, safety assurance, and recovery. Its essential timing requirements are the 100 ms heartbeat, 300 ms link-loss timeout, and 200 ms fallback bound. The state machine presents the same logic as operational states: normal platoon operation, communication loss, safe mode, emergency braking, safe-gap restoration, and conditional recovery.

The message sequence in Figure ?? shows the chronological behaviour. The lead truck emits heartbeats, the watchdog detects their absence, and the `link_lost` broadcast forces the coupled truck into independent driving. The alternative fragments distinguish recovery from continued link loss and show the emergency-braking branch for an unsafe gap. A leaving truck is also part of the platoon context. Its request is handled only when the lead truck is available, because the controller serialises join and leave requests and processes one request at a time.

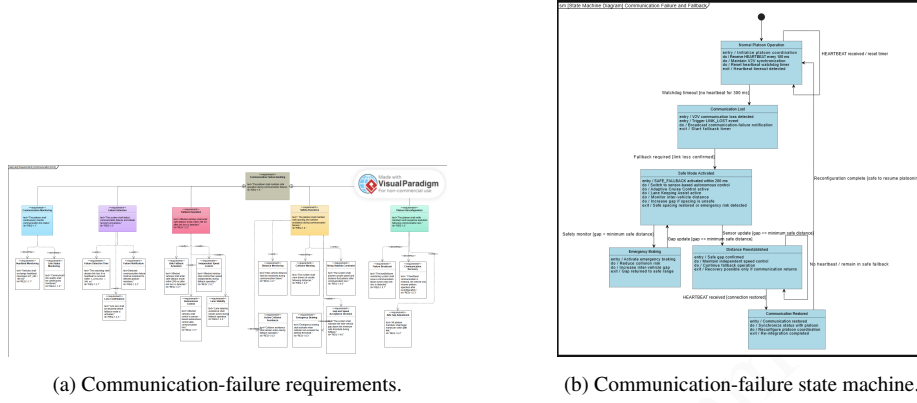


Fig. 7: SysML/UML models for Scenario 3.

5.2 UPPAAL Timed-Automata Model

The SysML/UML diagrams describe the system-level behaviour, while UPPAAL formalises the timing-critical subset. One UPPAAL time unit represents 10 ms. The principal constants are $T_{HB}=10$, $T_{LINKLOSS}=30$, and $T_{FALLBACK}=20$. The model contains a joining truck, a lead truck, two parameterised leaving trucks, and a watchdog. Handshake channels coordinate joining and leaving, whereas heartbeat and link_lost are broadcast channels.

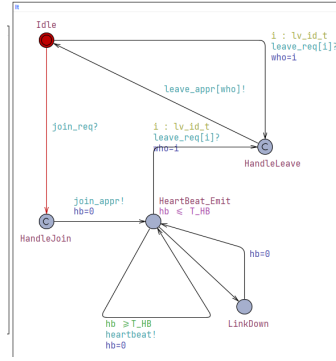
5.3 Lead Truck and Request Serialisation

The lead-truck automaton contains Idle, HandleJoin, HeartBeat_Emit, HandleLeave, and LinkDown. In HeartBeat_Emit, the invariant $hb \leq T_{HB}$ and the self-loop guard $hb \geq T_{HB}$ force a heartbeat every 100 ms. Moving to LinkDown stops transmission and models the communication failure; returning to HeartBeat_Emit models restoration.

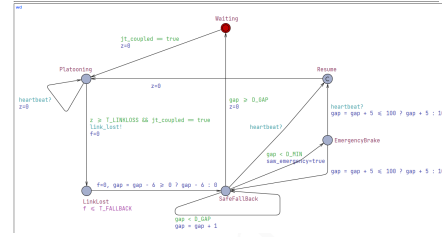
For leave requests, the lead stores the identifier of the truck currently being served in who. Because it enters the committed HandleLeave state and stores only one identifier, only one request is processed at a time. If both leaving trucks request departure together, UPPAAL nondeterministically accepts one; the other waits until the lead is available.

5.4 Watchdog and Fallback

The watchdog contains Waiting, Platooning, LinkLost, SafeFallback, EmergencyBrake, and Resume. Clock z measures the time since the last heartbeat and is reset on every heartbeat. When $z \geq T_{LINKLOSS}$ and the joining truck is coupled, the watchdog broadcasts



(a) Lead-truck automaton.



(b) Watchdog automaton.

Fig. 8: UPPAAL templates central to Scenario 3.

`link_lost!` and enters `LinkLost`. The invariant $f \leq T_{\text{FALLBACK}}$ bounds the transition to safe fallback.

If the abstract gap is below $D_{\min} = 10$ m, the watchdog enters `EmergencyBrake`; otherwise it increases the gap toward $D_{\text{gap}} = 50$ m. A returning heartbeat permits transition through `Resume`. This resumes monitoring, but the joining truck has already cleared its coupled flag, so complete re-coupling would require a new join procedure.

5.5 Verification and Results

Figure below shows the verifier output. The properties were kept compactly grouped according to their purpose:

- $A[]$ not deadlock verifies that the integrated processes never reach a global state in which no progress is possible.
- $E\langle\rangle$ `jt.PlatoonActive` and the departure reachability properties show that coupling and decoupling are executable. The corresponding waiting-state invariants verify the 100 ms approval bound.
- $lvX.SafetyVerify \dashv\dashv (lvX.Deported \parallel lvX.InPlatoon)$ verifies that each departure decision resolves rather than remaining blocked.
- $E\langle\rangle$ `wd.SafeFallback`, $A[]$ (`wd.LinkLost` imply `wd.f <= T_FALLBACK`), and `wd.LinkLost` $\dashv\dashv$ `wd.SafeFallback` jointly show that fallback is reachable, mandatory after link loss, and bounded by 200 ms.



Fig. 9: UPPAAL implementation and verification.

- $E \langle \rangle wd.EmergencyBrake$ confirms that the emergency path is executable. $E \langle \rangle wd.Resume$ together with $wd.Resume \rightarrow wd.Platooning$ verifies conditional recovery when communication returns.
- $E \langle \rangle (saw_couple \ \&\& \ saw_depart \ \&\& \ saw_emergency)$ provides one witness trace covering coupling, departure, and emergency fallback.

The stronger property $wd.LinkLost \rightarrow wd.Platooning$ is not expected to hold because the link may remain down indefinitely. This is a correct safety interpretation: the truck must remain in fallback rather than falsely resuming platoon control.

6 Limitations and Conclusion

The model abstracts continuous vehicle dynamics into discrete gap updates and does not represent packet corruption, sensor noise, or actuator behaviour. The watchdog is tied to the joining-truck coupling flag; the two leaving-truck templates do not independently receive the communication-failure broadcast. Recovery resumes watchdog monitoring but does not automatically re-couple the affected truck. These abstractions keep verification tractable while still testing the required timing, ordering, emergency response, and request-serialisation logic.

The annotation work produced consistent ground truth for the SVM lane-following pipeline, while the SysML/UML and UPPAAL work demonstrated how communication loss can be detected and handled safely. The verified model establishes deadlock freedom, bounded fallback, reachable emergency braking, and conditional recovery.

Declaration of Originality

We, Team 1, declare that this paper was prepared using only the stated sources and aids. Quoted or paraphrased material is identified accordingly. The work has not been submitted in the same form for another examination.